DAYANANDA SAGAR ACADEMY OF TECHNOLOGY & MANAGEMENT

(An Autonomous Institute under VTU)



Department of Information Science and Engineering

7th Semester-2026

Project Phase-2 Review-1

FLUXOR: Adaptive Reinforcement Learning-Based Load Balancing for Distributed Server Systems

1DT23IS102 : Nishanth Keekan

1DT23IS109 : Prateek Mangalgi

1DT23IS113 : Pritam Priyabrata Palai

Implementation status: RR / LC / RL benchmarking completed

Outline

- Introduction & motivation
  Problem statement and research gap
  Objectives
  RL methodology: state, action, reward and learning
  System architecture
  Implementation details
  Experimental setup
  Measured results: RR vs LC vs RL
  Conclusion and current status

Introduction

- Modern distributed applications must handle high request concurrency while maintaining low latency, availability and balanced resource utilisation.

  Traditional Round Robin (RR) and Least Connections (LC) make deterministic decisions using limited server information.

  FLUXOR uses a lightweight Deep Q-Network (DQN) that observes live infrastructure metrics and dynamically selects a backend server.

  The implementation combines a Go load balancer, three Node.js backends, a Python/PyTorch RL service, Prometheus monitoring and a React visualisation layer.

Problem Statements

Traditional load balancing algorithms make routing decisions without considering the complete real-time state of distributed servers.

This can cause:
- uneven workload distribution
- increased response and tail latency
- routing requests to overloaded servers
- degraded performance during workload changes

Research gap identified in the paper:
Most reviewed approaches are simulation/emulation based, GPU-dependent, offline-trained, single-objective, or lack real-time routing visualisation.

FLUXOR targets a CPU-feasible, online, multi-objective and containerised solution.

Objectives

- 1. Develop a lightweight DQN-based adaptive load balancer.

  2. Enable online learning from a cold start using live server metrics.

  3. Compare RL routing against Round Robin and Least Connections.

  4. Evaluate latency, tail latency, load distribution and overload behaviour.

  5. Provide real-time monitoring and visualisation of routing decisions.

Proposed Methodology

REINFORCEMENT LEARNING-BASED ROUTING

1. STATE

The agent observes:

- CPU utilisation
- Active connections
- Response latency
- Global request rate
- Queue depth

For 3 servers:

S = [cpu1, conns1, lat1, cpu2, conns2, lat2,
cpu3, conns3, lat3, rate, queue]

2. ACTION

A = {Server 1, Server 2, Server 3}

The selected action determines where the current request is routed.

Proposed Methodology

3. REWARD

The reward simultaneously considers:

Response latency
↓
CPU imbalance
↓
Overload prevention

$R(s,a) = -0.5(\text{Latency}/1000)$
$-0.3\sigma_{CPU}$
-2.0 Poverload

Poverload = 1 when selected-server CPU > 85%.

$\gamma = 0.95$

This encourages low latency while avoiding overloaded and poorly balanced routing decisions.

Proposed Methodology and Mathematical Model

4. ONLINE LEARNING LOOP

Observe live state
↓
Choose backend
↓
Process request
↓
Receive feedback
↓
Calculate reward
↓
Store transition / update DQN
↓
Improve future routing

Current implementation:
- PyTorch DQN
- Experience replay
- Target network
- ε-greedy exploration

MDP = (S, A, R, γ)

State

$$S = [CPU_1, C_1, L_1, ..., Rate, Queue]$$

Action

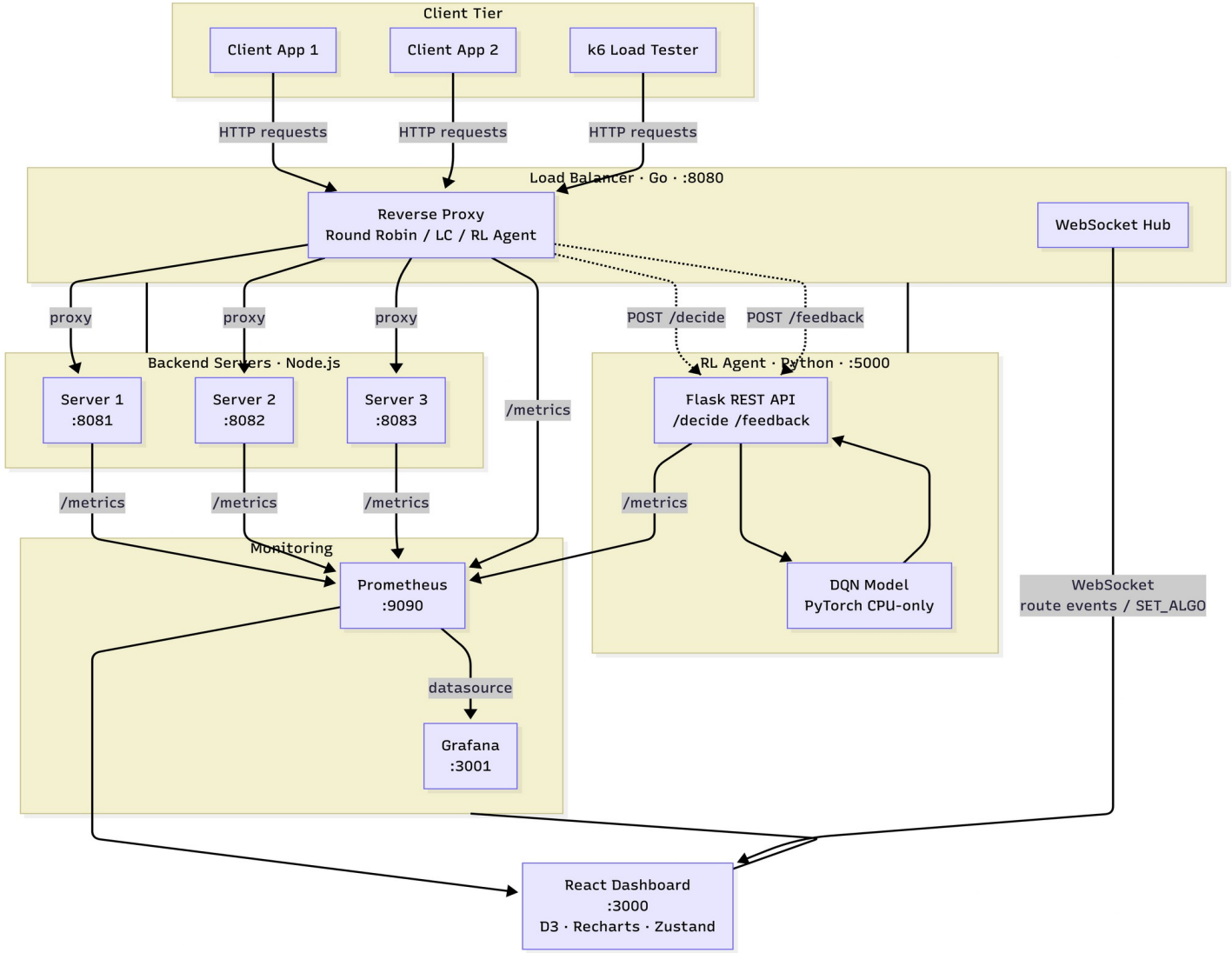
$$A = \{Server_1, Server_2, Server_3\}$$

Discount

$$\gamma = 0.95$$

$$R(s,a) = -0.5\frac{Latency}{1000} - 0.3\sigma_{CPU} - 2P_{overload}$$

FLUXOR SYSTEM ARCHITECTURE



Implementation Details

MONITORING

- Prometheus — metrics collection
- Grafana — monitoring dashboards

VISUALISATION

- React — dashboard
- D3.js — animated topology
- Recharts — time-series panels
- WebSocket — live routing events

OBSERVABILITY

Every routing decision can be traced from the load balancer to the RL agent and backend metrics.

Implementation Details

BACKEND & LOAD BALANCING

- Golang — reverse proxy / load balancer
- Node.js — three distributed backend servers
- Docker & Docker Compose — containerisation
- Runtime switching: RR / LC / RL

REINFORCEMENT LEARNING

- Python
- PyTorch
- Flask REST API
- Deep Q-Network
- /decide for routing decisions
- /feedback after request completion

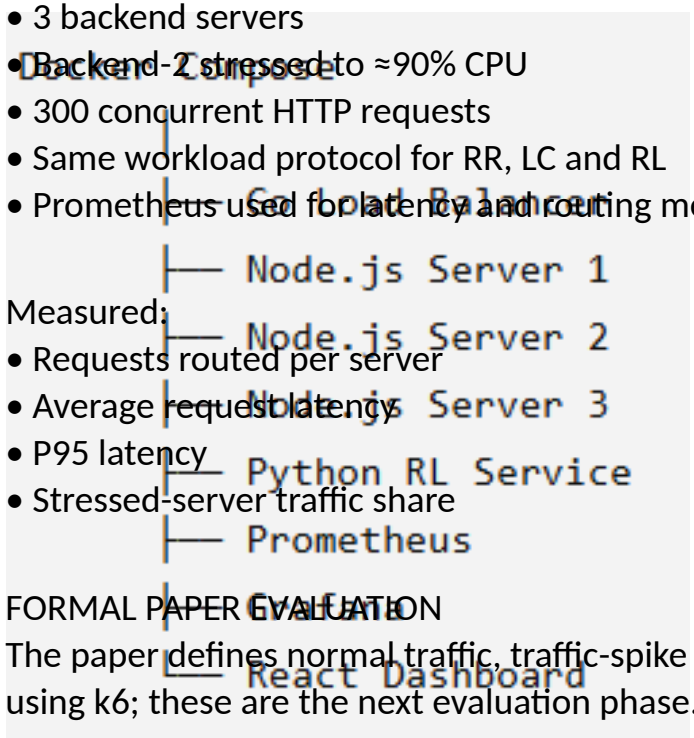
CURRENT IMPLEMENTATION STATUS

RR, LC and RL routing are operational and benchmarked under concurrent load.

EXPERIMENT SETUP

CURRENT COMPLETED EXPERIMENT

Scenario:
- 3 backend servers
- Backend-2 stressed to ≈90% CPU
- 300 concurrent HTTP requests
- Same workload protocol for RR, LC and RL
- Prometheus used for latency and routing metrics



Measured:
- Requests routed per server
- Average request latency
- P95 latency
- Stressed-server traffic share

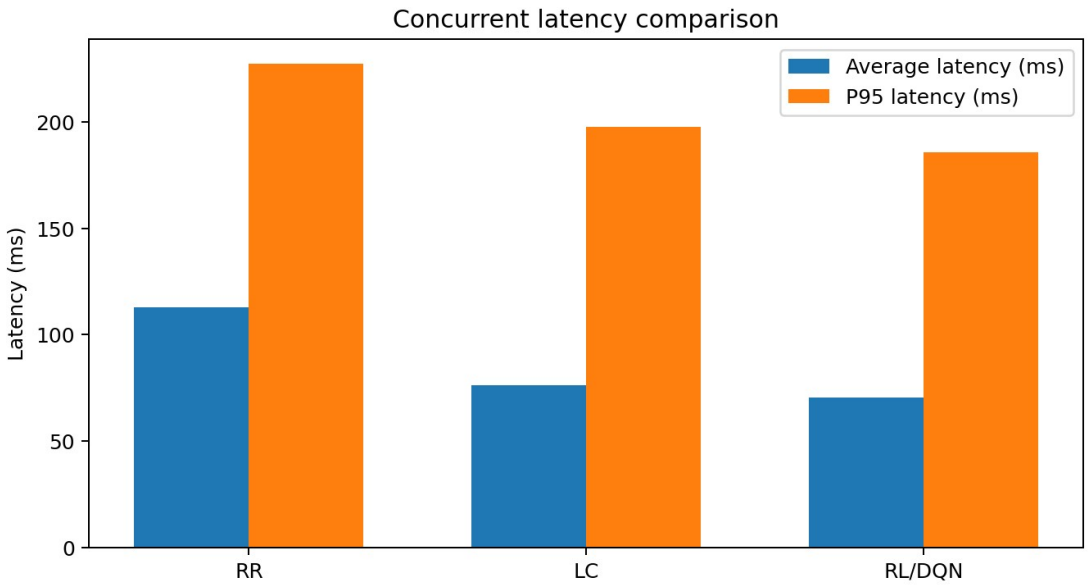
FORMAL PAPER EVALUATION

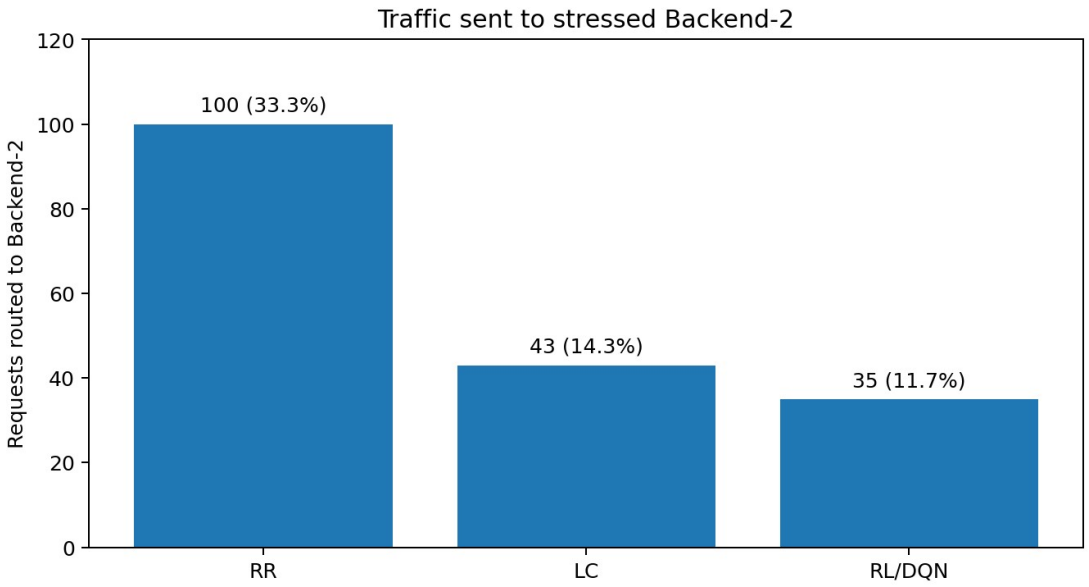
The paper defines normal traffic, traffic-spike and partial-server-failure scenarios using k6; these are the next evaluation phase.

Current validation: concurrent curl workload + Prometheus
Next phase: formal k6 multi-scenario evaluation

Scenario	Configuration	Purpose
Normal Traffic	50 users / 60 sec	Baseline performance
Traffic Spike	10 → 200 users	Test adaptability
Server Failure	50 users / 90 sec	Test fault handling

RESULTS — CURRENT MEASURED BENCHMARK





300 concurrent requests | Backend-2 ≈ 90% CPU

RR: 112.89 ms avg / 227.50 ms P95 | LC: 76.02 ms / 197.67 ms | RL: 70.48 ms / 185.71 ms

Traffic to stressed B2: RR 33.3% → LC 14.3% → RL 11.7%

Conclusion

- CURRENT CONCLUSIONS

  • RL adapts routing using real-time server conditions.
  • In the completed concurrent benchmark, RL achieved the lowest observed average latency: 70.48 ms.
  • RL also achieved the lowest observed P95 latency: 185.71 ms.
  • Traffic sent to the stressed backend fell to 11.7%, versus 14.3% for LC and 33.3% for RR.
  • The DQN operates as a CPU-oriented online routing service.
  • Prometheus and the dashboard provide visibility into routing and server state.

  NEXT PHASE
  Repeat experiments across multiple runs and scenarios, then optimise the RL state/reward/hyperparameters based on measured evidence.

FLUXOR — Research Paper

Paper basis: adaptive RL load balancing, literature survey, MDP formulation, system architecture and evaluation methodology.

Current implementation now includes measured RR vs LC vs RL concurrent results.

Submission / acceptance / presentation status: UPDATE BEFORE REVIEW

Measured project result — not projected paper target